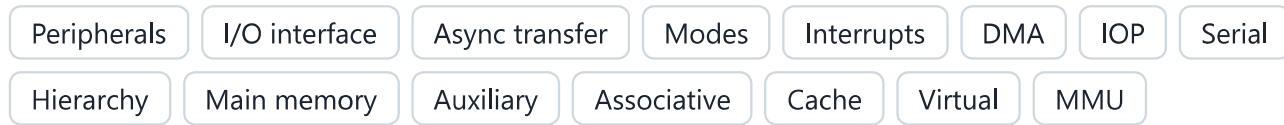


Unit V: Input-Output Organization and the Memory System

Notes with diagrams. Part A covers I/O, and Part B covers memory.



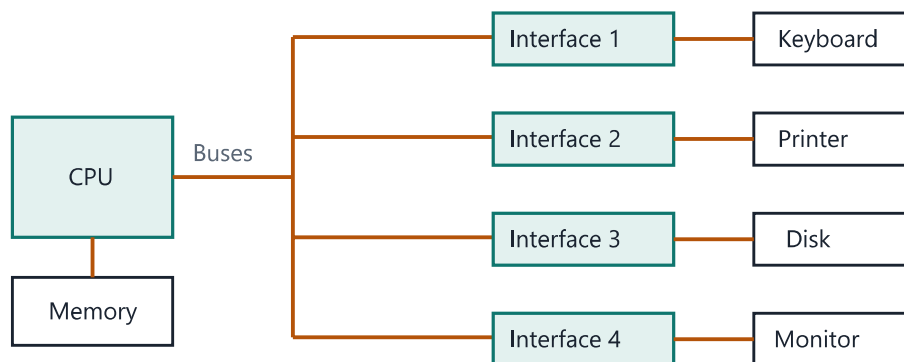
Part A: Input-output organization

1. Peripheral devices

Peripherals are external devices attached to the computer for input, output or storage. Examples: keyboard, mouse, scanner (input); monitor, printer (output); magnetic disk, tape, SSD (storage, also called auxiliary memory). They differ greatly in speed, data format and electrical signals from the CPU, so each needs an interface.

2. Input-output interface

An interface sits between the CPU buses and a peripheral. It resolves differences in **speed**, **data format** and **signal levels**, and supplies control and status information. Each interface contains registers (data, control/command, status) that the CPU treats as I/O ports.



Each interface has its own address and connects the common bus to one peripheral.

I/O bus and addressing of interfaces

The CPU selects an interface using the address bus, sends a function code (control, status, data output, data input) and transfers data on the data bus. Two schemes:

Scheme	How it works
Isolated I/O	Separate address space and separate IN/OUT instructions and control lines. Memory addresses stay free.
Memory-mapped I/O	Interface registers share the memory address space. Ordinary load/store instructions work on I/O, but some memory addresses are used up.

3. Asynchronous data transfer

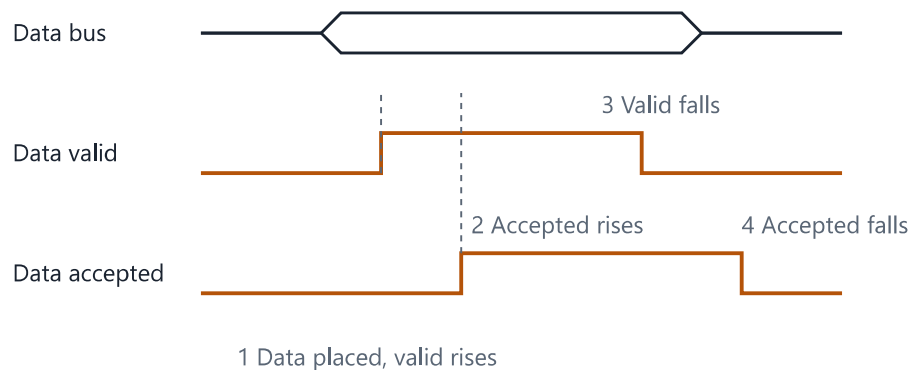
When the source and destination units have independent clocks, a control signal must tell when data is valid. Two methods are used.

Strobe control

One control line, the strobe, is used. In a **source-initiated** transfer, the source puts data on the bus and then pulses the strobe, and the destination latches the data on the strobe. In a **destination-initiated** transfer, the destination pulses the strobe to request data and the source supplies it. Weakness: the source cannot know the data was accepted (or the destination cannot know the data is valid).

Handshaking

Two lines give acknowledgment in both directions. In source-initiated handshaking: **data valid** goes from source to destination, and **data accepted** goes back. This guarantees reliable transfer between units of different speeds.



4. Modes of transfer

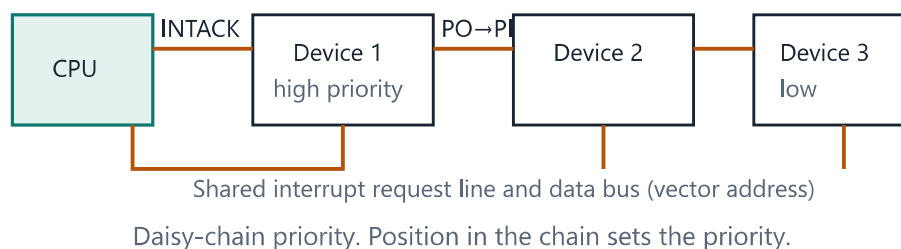
Mode	Description	Drawback / advantage
Programmed I/O	CPU runs a loop that polls the interface status flag and moves data.	CPU busy-waits and wastes time.
Interrupt-initiated I/O	Interface raises an interrupt when it is ready. CPU does other work until then, then runs a service routine.	Efficient, but each word costs an interrupt.
Direct memory access (DMA)	DMA controller moves blocks between I/O and memory without the CPU.	Best for fast, bulk transfers.

5. Priority interrupt

With many devices, the system must decide which interrupt to serve first, identify the source, and allow higher-priority devices to interrupt lower ones.

Daisy-chain (hardware serial priority)

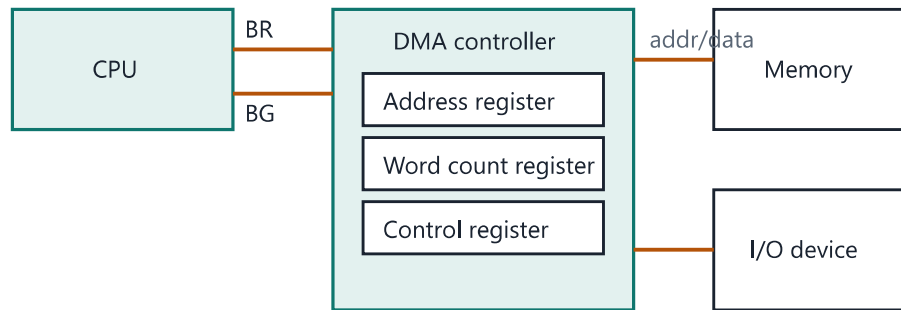
Devices are wired in a chain by priority. The CPU's interrupt acknowledge (INTACK) signal enters device 1. If a device is not requesting, it passes the signal on (PO to the next PI). The first requesting device blocks the signal and places its **vector address** on the bus.



Parallel priority uses an interrupt register with one bit per device, a mask register to enable or disable each device, and a priority encoder that outputs the vector for the highest-priority active bit. It is faster than daisy chaining but needs more hardware. **Polling** is a software method: one common routine checks each device in priority order.

6. Direct memory access (DMA)

DMA lets a peripheral exchange data with memory directly. The CPU is asked to give up the buses using **BR** (bus request) and answers with **BG** (bus grant). The DMA controller then drives the address and control lines.



Interrupt to CPU when the word count reaches zero

DMA controller with address, word count and control registers.

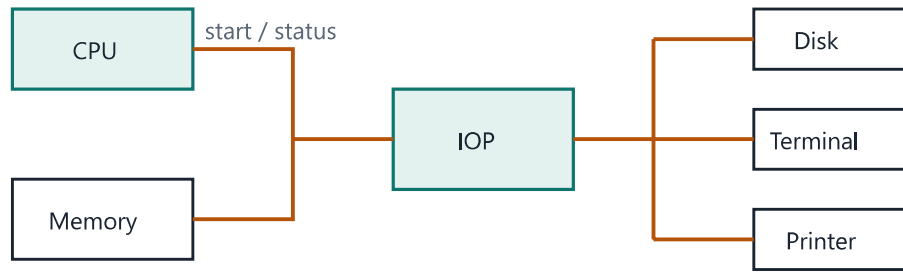
Transfer steps

The CPU loads the starting address and word count and starts the controller. The device requests DMA, the controller gets the bus, and each word is moved while the address register increments and the word count decrements. At zero, the controller interrupts the CPU.

DMA mode	Behaviour
Burst (block)	The controller keeps the bus until the whole block is sent. The CPU is idle meanwhile.
Cycle stealing	The controller takes the bus for one memory cycle per word, so the CPU is only slowed slightly.

7. Input-output processor (IOP)

An IOP is a processor with its own instruction set that handles I/O, so the CPU is relieved of I/O work. It goes beyond DMA by fetching and executing I/O programs (**channel programs**) that can also do format conversion, error checking and code translation. The CPU only starts the IOP and later handles a completion interrupt.



The CPU and IOP share memory. The IOP runs the I/O program and talks to devices.

Sequence: CPU checks the IOP is ready, sends a "start I/O" instruction pointing to the program in memory, and continues. The IOP fetches its commands, transfers the data, and sends an interrupt when done. The CPU then reads the IOP status word.

8. Serial communication

Serial transmission sends bits one after another on one line. It is cheaper than parallel over long distances. Modes: **simplex** (one direction), **half duplex** (both directions, one at a time), **full duplex** (both at once).

Type	Description
Asynchronous	Each character is framed by a start bit (0) and 1 or 2 stop bits (1). The line idles at 1. Clocks need not be synchronised, but overhead is high.
Synchronous	A shared or recovered clock sends blocks of characters with sync characters. It is faster with lower overhead.



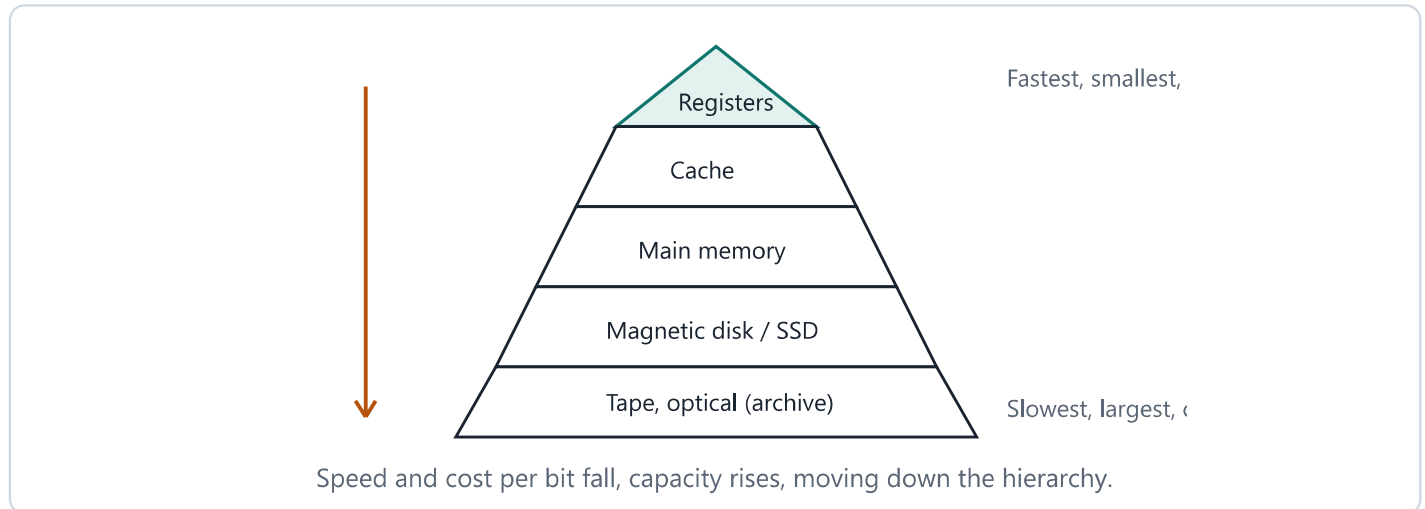
Asynchronous frame: start bit, data bits (LSB first), optional parity, stop bit.

A **UART** (asynchronous) or **USART** (also synchronous) interface converts parallel CPU data to serial with a transmitter shift register, and back with a receiver shift register. Baud rate is signalling rate; e.g. 10 characters/s at 11 bits each = 110 baud.

Part B: The memory system

9. Memory hierarchy

No single memory is fast, large and cheap. A hierarchy places small fast memory near the CPU and large slow memory further away. **Locality of reference** (programs reuse nearby addresses and recent data) makes this work.



Main memory talks to the CPU through cache. Auxiliary memory supplies bulk storage, and programs are brought into main memory when needed.

10. Main memory

RAM is read/write. **SRAM** uses flip-flops: fast, needs no refresh, costly (used for cache). **DRAM** stores charge in capacitors: dense and cheap but must be refreshed (used for main memory). **ROM** is non-volatile and holds the bootstrap loader. Types: PROM, EPROM (UV erasable), EEPROM and flash (electrically erasable).

Memory address map and chip connection

The designer assigns address ranges to RAM and ROM chips. Example: a 512-byte RAM chip needs 9 address lines. With 16 address lines, the upper lines go to a decoder that selects a chip. For four 128×8 RAM chips and one 512×8 ROM, address bits 8 and 9 select the chip (00 to 11 for RAM 1 to 4, and bit 10 = 1 selects ROM).

Component	Size	Address range (hex)	Lines used
RAM 1	128×8	0000 to 007F	7
RAM 2	128×8	0080 to 00FF	7
RAM 3	128×8	0100 to 017F	7

RAM 4	128×8	0180 to 01FF	7
ROM	512×8	0200 to 03FF	9

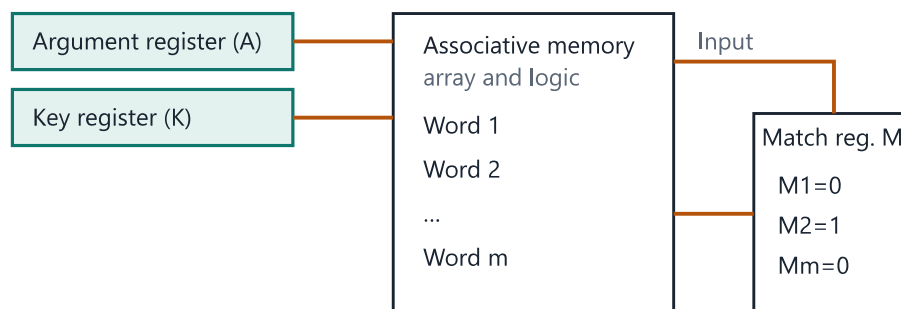
11. Auxiliary memory

Bulk, non-volatile, slow storage. **Magnetic disk:** platters divided into concentric *tracks* and pie-slice *sectors*. Access time = seek time (move head) + rotational latency (average half a rotation) + transfer time. **Magnetic tape:** sequential access, cheap, used for backup. **Optical disks** and **flash/SSD** are other forms. Data is recorded with schemes such as NRZ or phase encoding.

Example: 7200 rpm gives 8.33 ms per revolution, so average rotational latency = 4.17 ms.

12. Associative memory (CAM)

Content-addressable memory is accessed by **content**, not address. The argument register (A) holds the search word, the key register (K) masks which bits matter, and every word is compared in parallel. Matching words set bits in the match register (M), and the read logic outputs the matched word. It is used for fast table lookup such as the cache store and the TLB.



Each stored word is compared with A under mask K, and matches appear in M.

Example: A = 101 111100, K = 111 000000. Only the first three bits are compared, so a word 101 000001 matches.

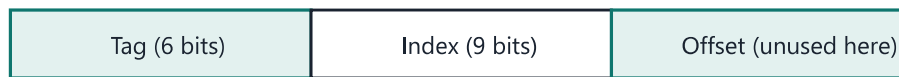
13. Cache memory

A small, fast memory between the CPU and main memory that holds recently used blocks. A **hit** means the word is in cache, and a **miss** means it must be fetched from main memory. **Hit ratio**

= hits / total accesses. **Average access time** = $h \cdot t_c + (1-h) \cdot t_m$ (simple model). Example: $h = 0.9$, $t_c = 100 \text{ ns}$, $t_m = 1000 \text{ ns}$ gives 190 ns .

Mapping

Mapping	Rule	Notes
Associative (fully associative)	Any block can go in any line. Address and data are stored, and the tag is the whole block address.	Most flexible, needs expensive CAM.
Direct	Line = block number mod number of lines. Address splits into tag index offset.	Simple and cheap. Conflicts occur when blocks share a line.
Set-associative	Block maps to a set (index), then goes in any of k lines in that set. Tags are compared within the set.	Good compromise (k-way).



Index picks the cache line. Tag is compared to the stored tag. Equal means hit.

Example: 32K word main memory (15-bit address), 512-word cache: tag = 6, index = 9.

Direct mapping address fields.

Replacement and write policy

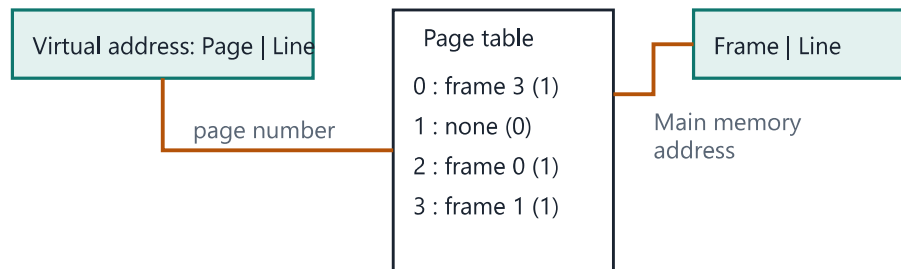
Replacement: FIFO, LRU (least recently used), random. **Write-through** writes to cache and main memory together (simple, slower). **Write-back** writes only to cache and marks the line dirty, and copies it to memory on replacement (faster, more complex).

14. Virtual memory

Virtual memory lets programs use a larger **logical (virtual) address space** than physical memory. The OS and hardware keep parts in main memory and the rest on disk and translate addresses on the fly. Virtual address space N is larger than physical space M .

Paging

Both spaces are split into equal fixed-size units: **pages** (virtual) and **page frames** (physical). The **page table** maps the page number to a frame number with a presence bit. If the presence bit is 0, a **page fault** occurs. The OS loads the page from disk, updates the table and restarts the instruction.



Presence bit 0 gives a page fault. The OS fetches the page from disk.

Page number indexes the page table. The frame number replaces it. The line (offset) is unchanged.

Item	Detail
Segmentation	Variable-size logical units (code, data, stack). Each has a base and limit. Good for protection and sharing.
Page replacement	FIFO (may show Belady's anomaly), LRU (good, costlier), optimal (theoretical best).
TLB	Small associative cache of recent page table entries. It avoids the extra memory access for translation.
Example	Address space 8K words, memory 4K words, page size 1K: virtual address = 13 bits (3 page + 10 line). The physical address is 12 bits (2 frame + 10 line).

15. Memory management hardware

A memory management unit (MMU) provides: **address mapping** (logical to physical, relocation), **protection** (prevent one program from accessing another's memory or writing to code) and **sharing** of common code.

- **Segmented-page mapping:** the logical address has segment | page | word. The segment table gives the page-table base for that segment, then the page table gives the frame. It combines the sharing and protection of segments with the fixed-size convenience of pages.
- **Numerical example:** 2 bits segment, 2 bits page, 8 bits word gives 12-bit logical addresses. Two table lookups plus one data access means 3 memory references, so a TLB is used to cut this to one.
- **Memory protection:** base and limit registers, or protection bits in page/segment table entries (read, write, execute, valid), and privileged mode for changing the mapping. A violation raises a trap.

- **Address translation by the MMU:** the CPU checks the TLB first (associative search on page number). A hit gives the frame directly. A miss walks the page table and updates the TLB.

Exam tip: for cache and paging numerical problems, first count address bits, then split them into fields. Use effective access time = $h \cdot t_{\text{cache}} + (1-h) \cdot t_{\text{memory}}$.